Project Summary

Project Summary: 16×16 Pipelined Wallace Tree Multiplier

1. Project Overview

This project implements a **16-bit × 16-bit unsigned binary multiplier** in Verilog using a **Wallace Tree** reduction architecture, combined with a **Carry Lookahead Adder (CLA)** for the final addition stage. The entire design is **pipelined into 5 register stages**, giving it a 4-cycle latency but a throughput of **one multiplication result per clock cycle** once the pipeline is full.

The design avoids using Verilog's built-in `*` operator anywhere in the actual hardware (DUT). Instead, multiplication is built up from first principles: bitwise AND gates (partial products), half adders, full adders (Wallace Tree compressors), and a carry-lookahead adder. The `*` operator is used **only** in the testbench, as a reference/golden model to check correctness.

Why a Wallace Tree?

A naive multiplier adds each partial product row sequentially (16 additions in series), which is slow. A Wallace Tree instead reduces all 16 partial product rows **in parallel**, column by column, using half adders (2-bit compressors) and full adders (3-bit compressors), cutting the number of rows down logarithmically across multiple "reduction levels" until only 2 rows remain per column. Those 2 final rows are then added with a single fast adder (the CLA) to produce the final product.

High-Level Data Flow

```
A (16b), B (16b)
      │
      ▼
[Stage 1] partial_product_gen   → 256 raw AND-bits, organized as 16 rows of 16 bits
      │  (pipeline register)
      ▼
[Stage 2] wallace_stage1        → Wallace reduction Levels 1 & 2  (16 bits/col → 8 bits/col max)
      │  (pipeline register)
      ▼
[Stage 3] wallace_stage2        → Wallace reduction Levels 3 & 4  (8 bits/col → 4 bits/col max)
      │  (pipeline register)
      ▼
[Stage 4] wallace_stage3        → Wallace reduction Levels 5 & 6  (4 bits/col → ≤2 bits/col)
      │  (pipeline register)
      ▼
[Stage 5] cla_32bit             → Final 32-bit Carry Lookahead Addition of the 2 remaining rows
      │  (pipeline register)
      ▼
P (32b), valid_out
```

Each of the 4 “register” boundaries above represents one clock cycle of pipeline latency, so a multiplication started at cycle T produces a valid result on `P` at cycle T+4. This was verified both analytically and through simulation (see Section 9).

---

2. File-by-File Breakdown

File	Role	Pipeline Stage
`half_adder.v`	Basic 2-bit compressor (building block)	used throughout
`full_adder.v`	Basic 3-bit compressor (building block)	used throughout
`partial_product_gen.v`	Generates all 256 partial product bits	Stage 1
`wallace_stage1.v`	Wallace reduction Levels 1 & 2	Stage 2
`wallace_stage2.v`	Wallace reduction Levels 3 & 4	Stage 3
`wallace_stage3.v`	Wallace reduction Levels 5 & 6	Stage 4
`cla_32bit.v`	Final 32-bit Carry Lookahead Adder	Stage 5
`pipelined_wallace_mult_16x16.v`	Top-level module wiring all stages + pipeline registers	All stages
`tb_pipelined_wallace_mult_16x16.v`	Self-checking testbench	Verification
`sim_output.txt`	Simulation log/results	Verification result

---

3. `half_adder.v` — Basic 2-bit Compressor

Module declaration:

```
        module half_adder (
            input  wire a,
            input  wire b,
            output wire sum,
            output wire cout
        );
```

Logic:

```
        assign sum  = a ^ b;   // XOR
        assign cout = a & b;   // AND
```

Purpose: Adds two single bits, producing a sum bit (stays in the same column) and a carry bit (moves to the next higher column). This is a “2:2 compressor” — used whenever exactly 2 partial-product bits remain in a column and need to be combined.

Truth table: | a | b | sum | cout | |—|—|—|—| | 0 | 0 | 0 | 0 | | 0 | 1 | 1 | 0 | | 1 | 0 | 1 | 0 | | 1 | 1 | 0 | 1 |

---

4. `full_adder.v` — Basic 3-bit Compressor

Module declaration:

```
module full_adder (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
```

Logic:

```
assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);
```

Purpose: This is the core "3:2 compressor" of the Wallace Tree. It takes 3 single bits from the same column and reduces them to 1 sum bit (stays in that column) + 1 carry bit (moves one column higher). This is what gives the Wallace Tree its speed advantage — instead of adding numbers sequentially, full adders compress 3 partial product bits into 2 in a single logic level, and this compression can happen in parallel across every column simultaneously.

Truth table: standard full-adder behavior — output is the count of 1's among (a, b, cin), encoded as a 2-bit binary number (cout = MSB, sum = LSB).

---

5. `partial_product_gen.v` — Stage 1: Partial Product Generation

Module declaration:

```
module partial_product_gen (
    input  wire [15:0] A,       // multiplicand
    input  wire [15:0] B,       // multiplier
    output wire [15:0] pp_row0, ..., pp_row15   // 16 output rows
);
```

Logic:

```
assign pp_row0  = A & {16{B[0]}};
assign pp_row1  = A & {16{B[1]}};
...
assign pp_row15 = A & {16{B[15]}};
```

Explanation: For long multiplication, each bit `B[i]` of the multiplier selects whether a full copy of `A` (shifted left by `i` positions) is added into the result. `{16{B[i]}}` is a Verilog replication operator that creates a 16-bit mask of all 1's (if `B[i]=1`) or all 0's (if `B[i]=0`). ANDing this mask with `A` produces row `i`: either an exact copy of A, or all zeros — this is the hardware equivalent of "if this bit of B is 1, copy A into this row; otherwise the row is 0."

Each row `i` is **not yet shifted** in the bus itself; instead, **row i's bit j is understood to belong to column (i + j)** of the final sum. This "virtual" shifting is handled implicitly when Stage 2 wires up which row/bit pairs feed into which column's adders — there is no separate shifting hardware.

This produces all 256 raw partial product bits (16 rows × 16 bits), which form the "Wallace Tree" matrix that needs to be reduced.

---

6. `wallace_stage1.v` — Stage 2: Wallace Reduction Levels 1 & 2

Module declaration:

```
module wallace_stage1 (
    input  wire [15:0] pp_row0 ... pp_row15,    // raw partial products
    output wire s2_col0_b0, s2_col1_b0, ..., s2_col31_b0   // 132 reduced output bits
);
```

Purpose: Performs the first two of six total Wallace reduction levels. Two reduction levels are grouped per pipeline stage (instead of registering after every single level) to balance combinational delay against the number of pipeline stages — this keeps latency reasonable (4 cycles total) while still meeting timing.

Structure: The file is organized strictly **column by column** (column 0 through column 31), and within each column, by **Level 1** first, then **Level 2**. Every column has a header comment stating: - How many input bits enter that column - How many `full_adder` and `half_adder` instances are used - How many bits simply pass through unchanged (when fewer than 2 bits are present, no compressor is needed)

Example for column 8 (Level 1): 9 input bits arrive at column 8 (combination of bits from several rows, since multiple rows can contribute to the same column). These are reduced using 3 full adders (3 FAs × 3 bits = 9 bits in, 3 sum bits stay in column 8, 3 carry bits move to column 9).

Naming convention for internal wires: - `w_l1_c{col}_fa{n}_s` / `_c` = Level 1, column `{col}`, full adder #n, sum/carry output - `w_l1_c{col}_ha{n}_s` / `_c` = Level 1, column `{col}`, half adder #n, sum/carry output - Same pattern with `l2` for Level 2

Output naming: `s2_col{c}_b{n}` = Stage 2 output, column `c`, bit `n` (a column can have multiple surviving bits since full reduction to ≤2 bits per column only completes after all 6 levels, spread across Stages 2–4).

Result: Reduces the maximum bit-count per column from 16 (widest column before reduction) down to a maximum of 8 bits per column.

---

7. `wallace_stage2.v` — Stage 3: Wallace Reduction Levels 3 & 4

Module declaration:

```
module wallace_stage2 (
    input  wire s2_col0_b0, ..., s2_col31_b0,     // 132 inputs from Stage 2
    output wire s3_col0_b0, ..., s3_col32_b0      // 79 outputs
);
```

Purpose: Identical structure and naming convention to `wallace_stage1.v`, but operates on the Stage-2 outputs and performs **Levels 3 and 4** of the Wallace reduction.

Progress so far (per the project's Wallace Tree Blueprint): | Point in pipeline | Max bits in widest column | |—|—| | Before any reduction | 16 | | After Stage 2 (L1+L2) | 8 | | After Stage 3 (L3+L4) | 4 | | After Stage 4 (L5+L6) | 2 (final target) |

Internal wire naming: `w_l3_c{col}_fa{n}_s/_c`, `w_l3_c{col}_ha{n}_s/_c` (Level 3), and `w_l4_...` (Level 4).

Output naming: `s3_col{c}_b{n}` — Stage 3 output, column `c`, bit `n`. Note that the column range has now extended to **column 32** (one wider than the 0–31 range of true 32-bit products), because carries from the widest columns of the partial-product matrix ripple one column further at each reduction level. This extra column is handled correctly (not dropped) but is proven to always evaluate to 0 for valid 16×16 multiplications.

---

8. `wallace_stage3.v` — Stage 4: Wallace Reduction Levels 5 & 6 (Final Reduction)

Module declaration:

```
module wallace_stage3 (
    input  wire s3_col0_b0, ..., s3_col32_b0,      // 79 inputs from Stage 3
    output wire s4_col0_b0, ..., s4_col33_b0       // outputs, ≤2 bits per column
);
```

Purpose: Performs the **last two** Wallace reduction levels (Level 5 and Level 6). After this stage, **every column holds at most 2 bits** — the defining condition that makes the result ready for a standard two-operand adder (the CLA in Stage 5).

Important note on output width: Because carries ripple one column further with each reduction level, this stage's outputs span **columns 0 through 33** (34 columns total), even though a valid 16×16 product never exceeds 32 bits (max value 65535 × 65535 = 4,294,836,225, which fits exactly in 32 bits). Columns 32 and 33 are therefore mathematically guaranteed to always be 0 for valid inputs — they are still wired correctly (never silently dropped) for design completeness, and the testbench implicitly confirms correctness across the full range of test cases.

Output naming and interpretation: `s4_col{c}_b{n}` — after this stage, each column has at most 2 bits. By convention: - Bit 0 of each column becomes part of **"Row A"** - Bit 1 of each column (if present) becomes part of **"Row B"**

These two rows (Row A and Row B) are the final two operands fed into the Stage 5 Carry Lookahead Adder.

---

9. `cla_32bit.v` — Stage 5: Final 32-bit Carry Lookahead Addition

This file contains **two modules**: `cla_32bit` (top-level 32-bit adder) and `cla_block_4bit` (a 4-bit building block).

9.1 Why Carry Lookahead instead of Ripple Carry?

A ripple-carry adder must wait for the carry to propagate one bit at a time from bit 0 to bit 31 — this is slow. A Carry Lookahead Adder (CLA) instead pre-computes: - **Generate** `g = a & b` — would this bit produce a carry regardless of carry-in? - **Propagate** `p = a ^ b` — would an incoming carry pass straight through this bit?

These signals let carries be computed directly via Boolean equations rather than rippling sequentially, dramatically reducing critical-path delay.

9.2 `cla_32bit` (top module)

```
module cla_32bit (
    input  wire [31:0] A_in,    // Wallace Tree Row A
    input  wire [31:0] B_in,    // Wallace Tree Row B
    output wire [31:0] Sum,     // final product P
    output wire        Cout     // unused for valid 16x16 multiply
);
```

Structure: Built from **8 chained 4-bit** `cla_block_4bit` **instances** (BLOCK0 through BLOCK7), covering bits [3:0], [7:4], ..., [31:28].

Logic flow: 1. Compute bitwise `g` and `p` for all 32 bits: `g = A_in & B_in`, `p = A_in ^ B_in`. 2. Each 4-bit block also produces a **block-level Generate (block_G)** and **block-level Propagate (block_P)** signal. 3. These block-level signals allow the carry **into** each 4-bit block to be computed directly (inter-block lookahead), without waiting for the previous block's internal bit-by-bit carry chain: `verilog    block_cin[1] = block_G[0] | (block_P[0] & block_cin[0]);    block_cin[2] = block_G[1] | (block_P[1] & block_cin[1]);    ... (and so on through block_cin[8], which becomes Cout)`

9.3 `cla_block_4bit` (sub-module)

```
module cla_block_4bit (
    input  wire [3:0] g, p,
    input  wire       cin,
    output wire [3:0] sum,
    output wire        block_G,
    output wire        block_P
);
```

Logic: Computes the three internal carries (`c1` , `c2` , `c3`) directly from `g` / `p` / `cin` via standard lookahead equations (not by rippling), so all 4 sum bits become available with the same short propagation delay:

```
c1 = g[0] | (p[0] & cin);
c2 = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);
c3 = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & cin);
sum[i] = p[i] ^ c[i-1] (with c0 = cin for bit 0)

block_G = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0]);
block_P = p[0] & p[1] & p[2] & p[3];
```

`block_G` = "would this whole 4-bit block produce a carry-out even with no carry-in?" `block_P` = "would this block propagate an incoming carry straight through?" (true only if *every* bit in the block propagates).

---

10. `pipelined_wallace_mult_16x16.v` — Top-Level Module

Module declaration:

```
module pipelined_wallace_mult_16x16 (
    input  wire        clk,
    input  wire        rst_n,      // active-low synchronous reset
    input  wire        valid_in,
    input  wire [15:0] A,
    input  wire [15:0] B,
    output reg  [31:0] P,          // final 32-bit product
    output reg         valid_out
);
```

10.1 What This Module Does

This is the **top-level wrapper** that wires together all 5 pipeline stages (`partial_product_gen` → `wallace_stage1` → `wallace_stage2` → `wallace_stage3` → `cla_32bit`), inserting a layer of registers between every pair of consecutive stages.

10.2 Pipeline Register Structure

- **`pp_row0_q ... pp_row15_q`** — registers holding the Stage 1 → Stage 2 boundary (raw partial products).
- **`s2_col*_q`** signals (132 of them) — registers holding the Stage 2 → Stage 3 boundary (after Wallace Levels 1+2).
- **`s3_col*_q`** signals (79 of them) — registers holding the Stage 3 → Stage 4 boundary (after Wallace Levels 3+4).
- **`s4_col*_q`** signals — registers holding the Stage 4 → Stage 5 boundary (after Wallace Levels 5+6).
- **`P`** — the final output register, holding the CLA's sum.

The `_q` suffix is a standard RTL naming convention meaning "the registered (clocked) version of this signal."

10.3 Valid Signal Pipeline

A chain of flip-flops (`valid_stage1` , `valid_stage2` , `valid_stage3` , `valid_stage4` , then `valid_out`) shifts `valid_in` through the same number of register stages as the data path, so `valid_out` lines up exactly with the clock cycle on which the corresponding result appears on `P` .

10.4 Stage Wiring (combinational logic between registers)

1. **Stage 1** (`partial_product_gen`): combinational, computes raw partial products from `A` , `B` .

2. **Stage 2** (`wallace_stage1`): combinational, takes `pp_row*_q` (registered) as inputs.
3. **Stage 3** (`wallace_stage2`): combinational, takes `s2_col*_q` (registered) as inputs.
4. **Stage 4** (`wallace_stage3`): combinational, takes `s3_col*_q` (registered) as inputs.
5. **Stage 5** (`cla_32bit`): combinational, takes `s4_col*_q` (registered) as inputs. The two final Wallace rows are concatenated into clean 32-bit buses:

```
cla_32bit U_CLA (
    .A_in({s4_col31_b0_q, ..., s4_col0_b0_q}),
    .B_in({s4_col31_b1_q, ..., s4_col7_b1_q, 1'b0 (x7 padding for low columns)}),
    .Sum(cla_sum), .Cout(cla_cout)
);
```

   Columns 32 and 33 are intentionally **not** connected to the CLA, since they are proven to always be 0 for valid 16×16 inputs (only columns 0–31 matter for the 32-bit product).

10.5 Reset Behavior

Reset (`rst_n`) is **active-low and synchronous**. When `rst_n = 0`, every pipeline register (all data registers, plus the 4 valid-signal registers and `P` itself) is cleared to 0 on the next clock edge. This guarantees the pipeline cannot output garbage data immediately after reset — it fully flushes the pipe.

10.6 Throughput and Latency

Because every stage is registered, a **new pair of operands (A, B) can be applied every single clock cycle** — the pipeline never needs to “drain” between operations. After an initial fill time of 4 cycles, it produces **one new result per clock cycle, every cycle** (verified by the testbench running 1000 back-to-back random multiplications).

Latency: A result for inputs applied at cycle T appears on `P` at cycle T+4 (5 register stages along the data path: `pp_row_q` , `s2_q` , `s3_q` , `s4_q` , and the final `P` register).

---

11. `tb_pipelined_wallace_mult_16x16.v` — Self-Checking Testbench

11.1 Purpose

This testbench fully verifies the DUT (`pipelined_wallace_mult_16x16`) by: 1. **Self-checking**: every output is automatically compared against a reference model (`A * B` , using Verilog’s built-in `*` operator — used **only** here, never in the actual design). 2. **Random + directed testing**: applies 12 directed “corner case” tests, then 1000 random `(A, B)` pairs. 3. **Latency-aware checking** via a **scoreboard** (a queue), since results don’t appear until 4 cycles after the corresponding inputs. 4. **Pass/fail summary** report at the end. 5. **Waveform dump** (`.vcd` file) for inspection in GTKWave.

11.2 Key Parameters

```
parameter CLK_PERIOD   = 20;     // 20 ns period = 50 MHz
parameter NUM_RANDOM   = 1000;   // minimum random test count
parameter PIPE_LATENCY = 4;      // verified pipeline latency in cycles
```

11.3 Clock Generation

```
initial clk = 1'b0;
always #(CLK_PERIOD/2) clk = ~clk;   // 50 MHz, 20ns period, 10ns high / 10ns low
```

The clock is generated **in the testbench**, not inside the DUT — the DUT itself is purely synchronous and works at any clock frequency the testbench supplies.

11.4 Scoreboard (Queue) Mechanism

```
        reg [15:0] scoreboard_A [0:NUM_RANDOM+50];
        reg [15:0] scoreboard_B [0:NUM_RANDOM+50];
        integer sb_write_ptr;   // next free slot to write a new sent input
        integer sb_read_ptr;    // next slot to read when a result comes back
```

Every input pair sent is pushed onto this queue (`send_input` task). Every time `valid_out` is high, the oldest unmatched entry is popped and compared (`check_output` task) — this correctly matches results to inputs **regardless of pipeline latency**, rather than naively comparing same-cycle inputs/outputs.

11.5 Key Tasks

- **`send_input(a_val, b_val)`**: drives `A`, `B`, `valid_in=1` for one clock cycle, and records the pair into the scoreboard.
- **`check_output`**: called every clock cycle; if `valid_out` is high, pops the oldest scoreboard entry, computes the expected product (`scoreboard_A[ptr] * scoreboard_B[ptr]`), and compares against `P` using `===` (exact bitwise compare).

11.6 Test Sequence

1. Apply synchronous reset for 3 cycles.
2. **Directed corner-case tests** (12 total): `0×0`, `max×max` (0xFFFF × 0xFFFF), `max×0`, `0×max`, `max×1`, `1×max`, half-max × max, alternating bit patterns (`0xAAAA × 0x5555`), `1×1`, near-max × near-max, etc. — these specifically target edge cases likely to expose carry-chain or overflow bugs.
3. Let the pipeline drain, then run **1000 random tests** back-to-back (one new input per clock cycle), which also demonstrates the 1-result-per-cycle throughput claim.
4. Drain the pipeline again, then print a final summary: total checked, total passed, total failed, and an overall PASS/FAIL verdict.

11.7 Safety Timeout

A separate `initial` block triggers `$finish` if the simulation runs unexpectedly long (e.g., if `valid_out` never asserts due to a wiring bug), to prevent the simulation from hanging forever.

11A. Line-by-Line Deep Dive: `tb_pipelined_wallace_mult_16x16.v` (with reasoning)

This section walks through the testbench almost line by line, explaining **what** each line does and **why** it was written that way — this is the part most likely to be questioned in a viva, because a testbench is where you prove your design is actually correct, not just "looks right."

11A.1 Timescale directive

```
        `timescale 1ns/1ps
```

- **What:** Sets simulation time unit = 1 ns, time precision = 1 ps.
- **Why:** Verilog simulators need an explicit time reference for all `#delay` statements. 1ns/1ps gives enough precision to express a 50 MHz (20ns) clock cleanly while still allowing finer rounding internally. Without this directive, `#10` would be ambiguous (10 of what unit?).

11A.2 Parameters

```
        parameter CLK_PERIOD   = 20;     // 20 ns -> 50 MHz
        parameter NUM_RANDOM   = 1000;   // minimum required random test count
        parameter PIPE_LATENCY = 4;      // verified pipeline latency in clock cycles
```

- **What:** Three configurable constants.
- **Why parameters instead of hardcoded numbers:** If the clock frequency, test count, or pipeline depth ever changes (e.g., if you add a pipeline stage later), you only edit these three lines instead of hunting through the whole file. This is standard good RTL/verification practice — "single source of

truth."

- **Why `PIPE_LATENCY = 4` specifically:** This number was *derived*, not guessed. The design has 5 register stages along the data path (pp_row_q, s2_q, s3_q, s4_q, and the final P register). An input applied at cycle T is captured into pp_row_q at the *next* edge (T+1), passes through stage 2's combinational logic and is captured into s2_q at T+2, then s3_q at T+3, then s4_q at T+4… and the CLA's combinational output (cla_sum) is captured into the final register P on that same edge. Tracing this carefully (and confirming it by simulation, which is exactly what was done) gives a measured latency of exactly 4 cycles from input to valid output. Notice the testbench does **not** hardcode reliance on the exact value of 4 anywhere in its checking logic (see 11A.6) precisely so that if this number were ever wrong, the testbench would still self-correct using the queue mechanism rather than silently mis-comparing results.

11A.3 DUT signal declarations and instantiation

```
reg         clk;
reg         rst_n;
reg         valid_in;
reg  [15:0] A;
reg  [15:0] B;
wire [31:0] P;
wire        valid_out;

pipelined_wallace_mult_16x16 DUT (
    .clk(clk), .rst_n(rst_n), .valid_in(valid_in),
    .A(A), .B(B), .P(P), .valid_out(valid_out)
);
```

- **What:** Declares testbench-side signals that drive/observe the DUT, then instantiates the actual multiplier as `DUT`.
- **Why `reg` for inputs, `wire` for outputs:** This is a hard Verilog rule, not a style choice. Anything the testbench *drives* with procedural assignment (inside `initial` / `always` blocks or tasks) must be declared `reg`. Anything the testbench only *reads*, which is driven by the DUT's internal logic, must be `wire`.
- **Why name the instance `DUT`:** "Device Under Test" — standard verification terminology, makes the testbench self-documenting (a reader instantly knows which signal set belongs to the thing being tested vs. the thing doing the testing).

11A.4 Clock generation

```
initial clk = 1'b0;
always #(CLK_PERIOD/2) clk = ~clk;
```

- **What:** Initializes clock low, then toggles it every `CLK_PERIOD/2` = 10 ns forever, producing a 20 ns period (50 MHz) square wave.
- **Why generate the clock in the testbench, not the DUT:** This is intentional and stated explicitly in the project's design notes. The DUT (`pipelined_wallace_mult_16x16`) is a purely synchronous design — every register inside it is sensitive only to `posedge clk`. It contains **no internal PLL, no internal frequency divider, no assumption about period**. This means the *same* DUT can be clocked at 50 MHz, 100 MHz, or 1 MHz just by changing this testbench's `CLK_PERIOD`, with zero changes to the DUT itself. This is good design practice: clock generation is a test/board-level concern, not something that belongs baked into reusable hardware IP.
- **Why `always #(CLK_PERIOD/2) clk = ~clk;` and not some other style:** This is the standard idiom for a free-running clock with 50% duty cycle in Verilog testbenches — toggle every half-period, forever (the `always` block has no sensitivity list other than the delay, so it just keeps re-triggering itself).

11A.5 VCD waveform dump setup

```
initial begin
    $dumpfile("wallace_mult_waveform.vcd");
    $dumpvars(0, tb_pipelined_wallace_mult_16x16);
end
```

- **What:** Tells the simulator to record every signal's value-over-time into a `.vcd` (Value Change Dump)

file.

- **Why:** This satisfies the project's "waveform generation" requirement — `.vcd` is the standard format readable by GTKWave (or any waveform viewer), letting you visually verify pipeline timing, e.g., literally watch `valid_in` go high and count exactly 4 clock edges until `valid_out` follows, as a second, independent visual confirmation of the latency claim (in addition to the automated pass/fail checking).
- **Why `$dumpvars(0, ...)`:** The `0` argument means "dump all signals recursively, at all levels of hierarchy" (i.e., not just top-level testbench signals, but everything inside `DUT` and inside the DUT's sub-modules too) — this gives full visibility for debugging if something fails.

11A.6 The scoreboard — why it exists and how it works

```
        reg  [15:0] scoreboard_A [0:NUM_RANDOM+50];
        reg  [15:0] scoreboard_B [0:NUM_RANDOM+50];
        integer     sb_write_ptr;
        integer     sb_read_ptr;
```

- **What:** Two parallel arrays act as a FIFO queue holding every `(A, B)` pair ever sent, plus two pointers: where to write the *next* sent pair, and where to read the *next* expected pair when a result comes back.
- **Why this is necessary (the core reasoning an IIT professor will likely probe):** Because this is a **pipelined**, not combinational, design, the value on `P` at any given clock cycle does **not** correspond to the `A`/`B` values present on the input ports during that *same* cycle — it corresponds to whatever `A`/`B` were applied **4 cycles earlier**. A naive testbench that did `if (P !== A*B) FAIL` on every cycle would report false failures constantly, because it would be comparing today's output against today's (wrong, not-yet-processed) input. The scoreboard solves this by **decoupling sending from checking**: every input is pushed onto a queue in the exact order it was sent, and every time a valid output appears, it is matched against the **oldest still-unconsumed** queue entry (FIFO ordering = "first input in, first result out", which is guaranteed because this is a simple linear pipeline with no reordering or stalls). This makes the testbench latency-agnostic: even if the pipeline depth were 4, 5, or 10 cycles, or even if it varied, the scoreboard would still correctly pair every result with its true corresponding input, because it's tracking *order*, not counting cycles.
- **Why arrays sized `[0:NUM_RANDOM+50]` and not just `[0:NUM_RANDOM]`:** A small safety margin (50 extra slots) is added to comfortably hold the 12 directed corner-case tests sent *before* the 1000 random tests, without risking an out-of-bounds array write.
- **Why two separate pointers (`sb_write_ptr`, `sb_read_ptr`) instead of one:** This is the standard circular-queue (FIFO) idiom: the write pointer always trails behind by however many results are still "in flight" inside the pipeline; the read pointer "catches up" each time a result is consumed. The gap between `sb_write_ptr` and `sb_read_ptr` at any moment exactly equals the number of in-flight, not-yet-completed multiplications.

11A.7 Pass/fail bookkeeping and the reference model

```
        integer total_checked;
        integer total_passed;
        integer total_failed;
        reg [31:0] expected_product;
```

- **What:** Running counters for the final summary, plus a register to hold the "golden" expected value for each check.
- **Why `expected_product` is computed with `*` only here:** This is the single most important correctness principle in the whole testbench. The DUT must **never** use Verilog's built-in multiply operator — if it did, you'd just be testing "does `*` equal `*`", which proves nothing about your Wallace Tree / CLA logic. By using `*` *exclusively* in the testbench (the "reference model" or "golden model"), the testbench acts as an independent, trusted oracle: it computes the answer a completely different way (using the simulator's built-in arithmetic, not your custom gate-level structure) and compares. Any mismatch proves a genuine bug in the Wallace Tree or CLA logic, not a tautology.

11A.8 `send_input` task

```
        task send_input(input [15:0] a_val, input [15:0] b_val);
            begin
                A         = a_val;
                B         = b_val;
                valid_in = 1'b1;
                scoreboard_A[sb_write_ptr] = a_val;
                scoreboard_B[sb_write_ptr] = b_val;
                sb_write_ptr = sb_write_ptr + 1;
                @(posedge clk);
            end
        endtask
```

- **Line-by-line:**
  - `A = a_val; B = b_val;` — drive the multiplicand/multiplier onto the DUT's input ports.
  - `valid_in = 1'b1;` — assert the valid flag so the DUT's pipeline registers know this cycle's `A`/`B` are real data, not garbage. (This matters because the DUT design explicitly supports gaps with `valid_in = 0`, e.g. during reset or pipeline drain — the valid signal is what's shifted through `valid_stage1..4` to eventually become `valid_out`.)
  - `scoreboard_A[...] = a_val; scoreboard_B[...] = b_val; sb_write_ptr = sb_write_ptr + 1;` — record this input into the queue **before** advancing time, guaranteeing the scoreboard always reflects exactly what was sent, in exact send order.
  - `@(posedge clk);` — block until the next rising clock edge, i.e., this task takes exactly one clock cycle to execute. This is what lets the main stimulus loop call `send_input` repeatedly to apply a new pair on every consecutive cycle (demonstrating full pipeline throughput).
- **Why bundle all of this into one task instead of writing it inline each time:** Encapsulation/reuse — this exact 6-line sequence is needed for every one of the 1012 test vectors; writing it as a task means it's written once and called many times, eliminating copy-paste bugs and making the stimulus section (Section 11A.11) read cleanly as a list of test cases.

11A.9 `check_output` task

```
        task check_output;
            begin
                if (valid_out) begin
                    expected_product = scoreboard_A[sb_read_ptr] * scoreboard_B[sb_read_ptr];
                    total_checked = total_checked + 1;
                    if (P === expected_product) begin
                        total_passed = total_passed + 1;
                        $display("PASS ...");
                    end
                    else begin
                        total_failed = total_failed + 1;
                        $display("FAIL ...");
                    end
                    sb_read_ptr = sb_read_ptr + 1;
                end
            end
        endtask
```

- **Line-by-line:**
  - `if (valid_out)` — only attempt a check when the DUT itself says `P` holds a genuine result. This guards against checking `P` during reset or pipeline-fill cycles, when `P` is meaningless (and `valid_out` is correctly low during exactly those cycles, by design of the DUT's valid-shift-register chain).
  - `expected_product = scoreboard_A[sb_read_ptr] * scoreboard_B[sb_read_ptr];` — pull the **oldest** still-pending input pair from the queue and compute its true product using `*`. This is the FIFO pop operation, paired with the golden reference computation.
  - `if (P === expected_product)` — compare using `===` (4-state exact match, including X/Z detection) rather than `==`. **Why `===` and not `==`:** `==` in Verilog returns `X` (unknown) if either operand contains any `X` or `Z` bits, so an `if` using `==` would already behave as false in that case too — but `===` is the explicit, rigorous choice here: it guarantees the comparison treats X/Z bits as real, intentional mismatches rather than something the test author overlooked. This is best practice for testbenches because it would also clearly catch a bug where the DUT outputs uninitialized (`X`) bits, e.g. an unreset register, with no ambiguity about why the test failed.

- `total_checked` / `total_passed` / `total_failed` increments — running counters for the final report.
- `$display(...)` — prints a human-readable PASS/FAIL line with the exact A, B, expected P, and actual P, with `$time` — this is exactly what appears in `sim_output.txt`, and is invaluable for debugging: if something fails, you immediately see which specific inputs caused it.
- `sb_read_ptr = sb_read_ptr + 1;` — advance the read pointer, "popping" this entry so it won't be re-checked.

- **Why this is a separate task instead of inline code in the clocked process:** Keeps the "what to do every cycle" logic isolated and readable, and means it can be called from exactly one place (Section 11A.10) without duplicating logic.

11A.10 The always block that drives checking every cycle

```
always @(posedge clk) begin
    check_output;
end
```

- **What:** An independent, always-running process that calls `check_output` on every single rising clock edge, for the entire simulation, regardless of what the stimulus process (Section 11A.11) happens to be doing at that moment.
- **Why this must be a separate, independent `always` block rather than being called manually after each `send_input`:** This is a subtle but important design decision. Because `valid_out` can go high at *any* clock edge — including cycles where the stimulus process is **not** calling `send_input` at all (e.g., during the "drain" period after the last input has been sent, or during the directed-test-to-random-test transition gap) — checking must happen on *every* cycle unconditionally, decoupled from the stimulus timing. If checking were only invoked from inside `send_input`, any result that became valid during a gap (when no new input is being sent) would be silently missed and never checked. This always-running watchdog process guarantees **zero results are ever dropped**, regardless of stimulus gaps.

11A.11 Main stimulus: `initial` block

Initialization:

```
sb_write_ptr  = 0;
sb_read_ptr   = 0;
total_checked = 0;
total_passed  = 0;
total_failed  = 0;
valid_in      = 1'b0;
A = 16'b0; B = 16'b0;
```

- **Why:** All bookkeeping/state must be explicitly initialized to known values at time 0. Verilog `reg` / `integer` variables default to `X` (unknown) if not initialized, which would corrupt the very first scoreboard comparisons. `valid_in = 0` and `A=B=0` ensure the DUT sees a clean, defined, "no input yet" state before reset is even applied.

Reset sequence:

```
rst_n = 1'b0;
repeat (3) @(posedge clk);
rst_n = 1'b1;
@(posedge clk);
```

- **What:** Holds reset active (`rst_n=0`) for 3 full clock cycles, then releases it, then waits one more cycle before sending any real stimulus.
- **Why 3 cycles and not just 1:** A single reset pulse is technically enough for this purely-synchronous-reset design (every register clears on the very next edge while `rst_n=0`), but holding it for multiple cycles is standard conservative practice — it guarantees the reset is robustly observed and cleanly separates "simulation startup noise" (where `X` values are still settling) from the start of actual testing.
- **Why one extra `@(posedge clk)` after releasing reset:** Gives one clean cycle for all registers to be definitively known-zero and stable before the very first `send_input` call, avoiding any race between the reset de-assertion and the first input being latched.

Directed corner-case tests:

```
        send_input(16'd0, 16'd0);
        send_input(16'hFFFF, 16'hFFFF);
        send_input(16'hFFFF, 16'd0);
        send_input(16'd0, 16'hFFFF);
        send_input(16'hFFFF, 16'd1);
        send_input(16'd1, 16'hFFFF);
        send_input(16'h8000, 16'hFFFF);
        send_input(16'hFFFF, 16'h8000);
        send_input(16'hAAAA, 16'h5555);
        send_input(16'h5555, 16'hAAAA);
        send_input(16'h0001, 16'h0001);
        send_input(16'hFFFE, 16'hFFFE);
```

- **What:** 12 hand-picked "directed" / corner-case tests, each chosen for a specific reason (this is the kind of reasoning a professor will specifically want to hear):
  - `0 × 0` — the absolute minimum case; tests that all-zero partial products correctly reduce to an all-zero output (no spurious carries generated from nothing).
  - `0xFFFF × 0xFFFF` (65535 × 65535) — the absolute **maximum** case; produces the largest possible 32-bit product (4,294,836,225) and stresses **every single adder and every carry chain in the entire design simultaneously** (every partial-product bit is 1). If there's any column miscount, any missing full/half adder, or any carry not wired correctly anywhere in the 6 Wallace levels or the CLA, this is the test most likely to catch it.
  - `0xFFFF × 0` and `0 × 0xFFFF` — tests that a fully-populated multiplicand/multiplier correctly zeroes out when multiplied by zero (catches bugs where some AND gate is wired wrong and a stray bit leaks through).
  - `0xFFFF × 1` and `1 × 0xFFFF` — "identity-like" cases; the product should exactly equal the non-1 operand. Also tests symmetry: confirms the design correctly handles the multiplicand and multiplier in *either* order (important because A and B play structurally different roles in partial-product generation — B selects which rows are nonzero, A is what's copied — so testing both orders independently verifies neither role has a hidden asymmetric bug).
  - `0x8000 × 0xFFFF` and `0xFFFF × 0x8000` — `0x8000` = binary `1000000000000000`, i.e., only the **most significant bit** set. This specifically stresses the carry chain in the **middle/high columns** of the Wallace Tree and the CLA's higher-order 4-bit blocks, which lower-value tests might not exercise as thoroughly.
  - `0xAAAA × 0x5555` and `0x5555 × 0xAAAA` — alternating bit patterns (`1010...` and `0101...`). These are classic "worst-case toggling" test vectors in digital design because they maximize the number of distinct 1-bit/0-bit transitions across columns, stressing the compressor logic (full/half adders) differently from either the all-1s or all-0s cases — likely to expose any column where the wrong AND-bit was wired into the wrong full/half adder input.
  - `1 × 1` — the smallest possible *nonzero* product; verifies the design doesn't have an off-by-one bug that breaks only at the very low end.
  - `0xFFFE × 0xFFFE` (65534 × 65534) — "near-max × near-max"; close to the maximum case but not identical, catching any bug that might coincidentally cancel out only in the very specific all-ones case.
- **Why directed tests are run *before* random tests, not after or interleaved:** Corner cases are the *most likely* to reveal genuine logic bugs (they exercise extremes that random testing might only hit rarely by chance), so it makes sense to check them first, get an early, easily-readable PASS/FAIL signal on the most important cases, before spending time on bulk random coverage.

Drain after directed tests:

```
        valid_in = 1'b0;
        repeat (PIPE_LATENCY + 2) @(posedge clk);
```

- **What:** Stops sending new inputs and waits `PIPE_LATENCY + 2` = 6 cycles.
- **Why:** This lets the 12 directed-test results, which are still working their way through the 4-cycle pipeline, fully drain out and get checked by the always-running `check_output` watchdog, **before** the random-test loop begins. The `+2` margin (beyond the bare minimum of 4) is a safety buffer to be certain

every in-flight result has definitely emerged before moving on. It also makes the console log far easier to read, since it cleanly separates “directed test results” from “random test results” in the printed output.

Random test loop:

```
        for (i = 0; i < NUM_RANDOM; i = i + 1) begin
            rand_a = $random;
            rand_b = $random;
            send_input(rand_a, rand_b);
        end
```

- **What:** Sends 1000 random `(A, B)` pairs, **one per clock cycle, back-to-back, with `valid_in` held high every cycle** (since `send_input` always sets `valid_in=1`).
- **Why exactly 1000 and not fewer:** This is a stated project requirement (“at least 1000 random test cases”) — large enough that, combined with the directed corner cases, it gives strong statistical confidence that no column or bit-position in the 16×16 multiplier matrix has a wiring bug, while still completing in a reasonable simulation time.
- **Why `$random` and not some fixed seed/pattern:** `$random` produces pseudo-random 32-bit values (truncated to 16 bits via the `reg [15:0] rand_a` assignment) on each call, giving broad, unpredictable coverage of the input space — this is far more likely to stumble onto an obscure bug (e.g., a single mis-wired bit in column 23) than a small hand-picked set ever could, simply through sheer combinatorial coverage.
- **Why this loop is *also* an implicit throughput test:** Because `send_input` is called every cycle with no gaps (`valid_in` stays high continuously for all 1000 iterations), this loop simultaneously proves the pipeline’s **one-result-per-cycle throughput** claim — if the design had any structural hazard (e.g., a stage that wasn’t truly pipelined and needed to “wait”), this back-to-back stress test would immediately desynchronize the scoreboard and start failing.

Final drain:

```
        valid_in = 1'b0;
        repeat (PIPE_LATENCY + 5) @(posedge clk);
```

- **What:** Stops new input, then waits `PIPE_LATENCY + 5` = 9 cycles.
- **Why:** Identical reasoning to the earlier drain — the *last* few of the 1000 random inputs are still inside the pipeline (up to 4 cycles’ worth) when the loop ends, so the simulation must keep running (and keep calling `check_output` via the always-block) until every single one of those final results has emerged and been checked. Without this, the very last handful of test vectors would never be verified, and `total_checked` would be less than 1012, silently weakening the test’s coverage claim.

Final summary and verdict:

```
        $display("..."); // final summary block
        if (total_failed == 0 && total_checked >= NUM_RANDOM) begin
            $display(" OVERALL RESULT        : PASS");
        end else begin
            $display(" OVERALL RESULT        : FAIL");
        end
        $finish;
```

- **What:** Prints final statistics and a single overall verdict, then ends simulation.
- **Why the verdict condition checks *both* `total_failed == 0` *and* `total_checked >= NUM_RANDOM`:** This is a deliberately defensive double-check. Checking only `total_failed == 0` is not actually sufficient on its own — if, for example, a wiring bug caused `valid_out` to never assert at all, then `check_output` would simply never run, `total_failed` would stay at 0 *by default* (zero comparisons made = zero failures), and a naive testbench would wrongly report PASS despite the design being completely broken. Requiring `total_checked >= NUM_RANDOM` additionally guarantees that a *meaningful number* of comparisons actually happened — a “PASS” is only reported if the design was both (a) never observed to produce a wrong answer, and (b) actually exercised by at least the required volume of test vectors. This is a textbook example of a self-checking testbench guarding against “false positive” passes caused by a silently broken `valid_out` or scoreboard chain.

- **Why `$finish` and not just letting the simulation run out:** `$finish` is the standard Verilog system task to cleanly terminate simulation once all intended work is done, rather than relying on the simulator hitting an arbitrary time limit.

11A.12 Safety timeout

```
        initial begin
            #(CLK_PERIOD * (NUM_RANDOM + 100));
            $display("*** TIMEOUT: simulation ran too long ... ***");
            $finish;
        end
```

- **What:** A completely separate, parallel `initial` block that waits for a generously long but bounded amount of simulated time (`CLK_PERIOD × (NUM_RANDOM+100)` ns), and if the main stimulus block hasn't already called `$finish` by then, forcibly ends the simulation with a timeout message.
- **Why this is necessary:** If the DUT has a serious bug — for instance, `valid_out` never asserts at all, or the pipeline somehow gets "stuck" — the main stimulus/check process would otherwise loop or wait forever (e.g., stuck inside the drain `repeat` loops waiting on clock edges that never produce the expected behavior), and the simulation would hang indefinitely, wasting compute time and giving no useful diagnostic output. This watchdog guarantees the simulation **always terminates** within a bounded time and clearly reports *that* it timed out (rather than just silently hanging), which is itself a useful debugging signal ("the design never produced valid_out — go look at the valid-shift-register chain").
- **Why this *particular* duration:** It must comfortably exceed the time needed for the entire intended test sequence (reset + 12 directed tests + drain + 1000 random tests + drain) under normal correct operation, with generous margin (`+100` extra cycles), so that it never falsely triggers on a working design, but still fires promptly enough on a broken one to avoid wasting excessive simulation time.

11A.13 Why this overall testbench architecture is the "right" one (summary reasoning)

If asked directly *"why did you design the testbench this way and not some simpler way?"*, the core argument is: 1. **A pipelined design cannot be checked cycle-by-cycle naively** — the scoreboard/queue is not optional complexity, it is the *minimum necessary mechanism* to correctly verify any pipelined datapath, because input and output are temporally decoupled. 2. **Self-checking removes human error from verification** — rather than a person eyeballing waveforms, an independent golden reference (`*`) computes the truth automatically, scales to thousands of vectors, and is far more reliable. 3. **Directed + random testing together give both depth and breadth** — directed tests target known-risky corner cases (all-ones, all-zeros, alternating patterns) with 100% certainty of hitting them; random testing adds broad statistical coverage across the full input space that hand-picking could never practically achieve. 4. **Independent, always-running checking avoids dropped results** — decoupling the check process from the stimulus process (Section 11A.10) guarantees no valid output is ever missed, regardless of what the stimulus is doing at that moment. 5. **A bounded, watchdog-protected simulation is mandatory engineering hygiene** — without the timeout (11A.12), any latent bug that breaks the valid/handshake logic would hang the simulation rather than failing loudly and immediately. 6. **The strict pass condition (11A.11) prevents false positives** — checking both zero failures *and* a minimum number of checks performed closes the loophole where a totally broken design (that never asserts `valid_out`) could otherwise be misreported as passing.

Together, this is exactly the pattern professional verification engineers use for any pipelined RTL block — and is the strongest answer to "how do you know your multiplier is actually correct?": it isn't just "it compiled" or "I checked one example by hand," it's 1012 independently-verified, latency-correctly-matched results with zero mismatches.

---

12. `sim_output.txt` — Simulation Results

This is the actual console log from running the testbench. Key results:

- Clock period confirmed: **20 ns (50.0 MHz)**.

- Pipeline latency confirmed: **4 clock cycles**.
- All **12 directed corner-case tests passed**, including the maximum-value case: `A=65535, B=65535 → expected P=4294836225, got P=4294836225` (correctly matches $65535^2 = 4{,}294{,}836{,}225$, the largest possible 16×16 product).
- All **1000 random tests passed** (each printed line shows matching expected/actual products).

Final summary block:

```
============================================================
 TEST SUMMARY
============================================================
 Total results checked : 1012
 Passed                : 1012
 Failed                : 0
 OVERALL RESULT        : PASS
============================================================
```

This confirms the design is **functionally correct across all directed and random test vectors**, with zero mismatches out of 1012 results checked.

---

13. Summary of Key Design Concepts (for explaining the project)

1. **Multiplication = AND array + addition.** `partial_product_gen` builds the AND array (16×16 = 256 bits).
2. **Wallace Tree compresses faster than sequential addition.** Instead of adding 16 rows one at a time, full adders (3:2) and half adders (2:2) reduce all columns of the matrix in parallel, level by level, until each column has ≤2 bits. This takes **6 reduction levels** total for a 16-bit operand, split across 3 pipeline stages (2 levels each) for balanced timing.
3. **Carry Lookahead Adder finishes the job fast.** The final 2-row addition uses a CLA (not ripple-carry) to avoid a slow 32-bit carry chain, using block-level Generate/Propagate signals for fast inter-block carry computation.
4. **Pipelining trades latency for throughput.** By registering between each of the 5 stages, the design accepts a new (A, B) pair every cycle and outputs one result every cycle (after a 4-cycle fill delay) — ideal for high-throughput applications like DSP or graphics pipelines.
5. **Verification via self-checking testbench + scoreboard.** Since pipelined results don't return immediately, a scoreboard queue is essential to correctly match each result to its corresponding input, and the design was validated against 1012 test vectors (12 directed + 1000 random) with zero failures.